

Adaptive Automotive C++ Hackathon

Title

Adaptive Smart Cabin & Vehicle Health Monitoring System

1. Hackathon Overview

Duration

15 Hours

Target Audience

Visteon Freshers (C++ Training Participants)

Objective

Design and implement a simplified Adaptive Automotive Application using modern C++ concepts covered during the training.

The application simulates an intelligent vehicle cockpit ECU which:

- Monitors vehicle health
- Manages cabin alerts
- Processes sensor data
- Performs event logging
- Supports multithreaded real-time simulation
- Uses modern C++ and STL-based architecture

The hackathon is intentionally aligned with automotive middleware and Adaptive AUTOSAR style software thinking:

- Service-oriented architecture mindset
 - Modular class design
 - Thread-safe data processing
 - Event-driven behavior
 - Resource ownership using RAII
 - Runtime polymorphism
 - Safe memory management
-

2. Problem Statement

Scenario

A next-generation vehicle cockpit domain controller is responsible for collecting data from multiple vehicle sensors and providing driver notifications.

Your team must develop a **Smart Cabin & Vehicle Health Monitoring System** in C++.

The system shall continuously:

- Read simulated sensor values
- Detect abnormal vehicle conditions
- Generate driver alerts
- Log critical events
- Maintain statistics
- Process multiple sensor streams concurrently

The application should mimic the behavior of an Adaptive Automotive application running on a high-performance ECU.

3. Functional Requirements

3.1 Sensor Simulation Module

The system shall simulate the following sensors:

Sensor	Example Data
Engine Temperature Sensor	85°C
Battery Voltage Sensor	12.5V
Vehicle Speed Sensor	72 km/h
Door Status Sensor	OPEN/CLOSED
Seatbelt Sensor	LOCKED/UNLOCKED
Tire Pressure Sensor	30 PSI

Requirements

- Each sensor must be represented as a class.
- Use inheritance and polymorphism.
- Common sensor interface shall be created.
- Sensor values should update periodically.

- Random value generation can be used.

3.2 Alert Management Module

Generate alerts when abnormal conditions occur.

Example Conditions

Condition	Alert
Engine temperature > 110°C	CRITICAL ENGINE OVERHEAT
Battery voltage < 10V	LOW BATTERY WARNING
Tire pressure < 25 PSI	LOW TIRE PRESSURE
Speed > 120 km/h	OVERSPEED WARNING
Door open while speed > 10 km/h	DOOR OPEN WARNING
Seatbelt unlocked while moving	SEATBELT WARNING

Requirements

- Use enum class for alert severity.
- Use operator overloading for formatted alert printing.
- Store active alerts in STL containers.

3.3 Vehicle Dashboard Module

Display:

- Current sensor values
- Active alerts
- Alert history
- Vehicle statistics

Requirements

- Use stream operators.
- Provide formatted output.
- Support file logging.

3.4 Event Logger Module

The application shall:

- Log critical alerts to a file

- Maintain timestamped history
- Support searching previous events

Requirements

- Use file streams.
- Use STL algorithms.
- Use lambda expressions for filtering.

3.5 Threading Module

The system must run multiple activities concurrently.

Example Threads

Thread	Responsibility
Sensor Thread	Periodically updates sensors
Monitoring Thread	Evaluates alert conditions
Dashboard Thread	Displays system state
Logger Thread	Writes events to file

Requirements

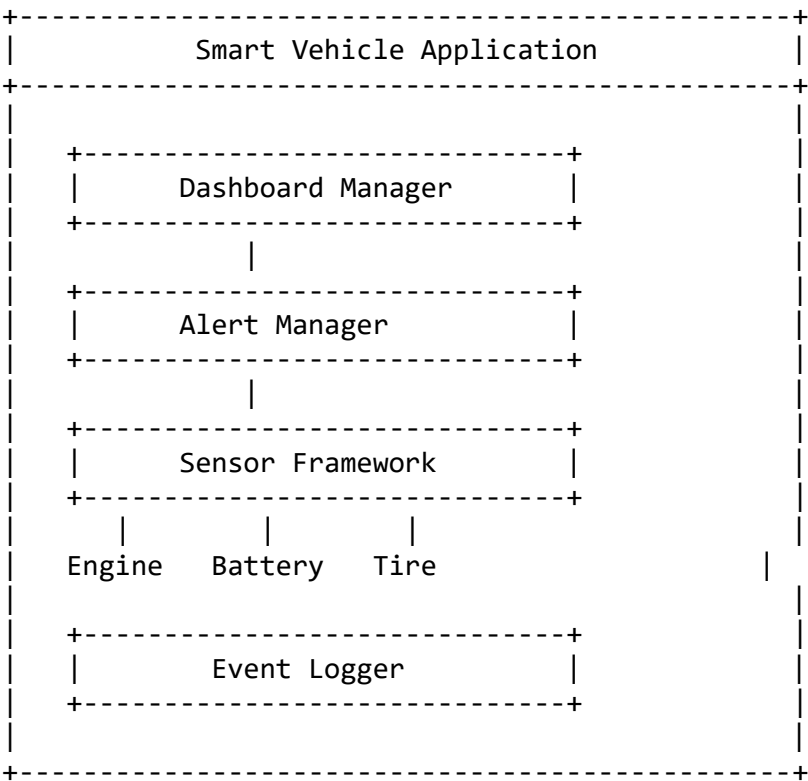
- Use `std::thread`
- Use mutex and `lock_guard`
- Prevent race conditions
- Use thread-safe shared data access

4. Mandatory C++ Concepts to be Used

Topic	Mandatory Usage
Classes & Objects	Sensor, Alert, Dashboard
Constructors/Destructors	Resource management
RAII	File handling / ownership
Inheritance	Base Sensor class
Virtual Functions	Runtime polymorphism
STL Containers	vector, map, set
Templates	Generic utility/logger
Smart Pointers	unique_ptr/shared_ptr
Exception Handling	File failures / invalid data

Topic	Mandatory Usage
Threads & Mutex	Concurrent execution
Lambdas	Alert filtering
Operator Overloading	Alert formatting
Static Members	Alert count / object tracking
Copy/Move Semantics	Event handling objects

5. Suggested Software Architecture



6. Recommended Class Design

Base Classes

```

class Sensor
{
public:
    virtual void update() = 0;
    virtual void display() const = 0;

```

```
    virtual ~Sensor() {}  
};
```

Derived Classes

- EngineTemperatureSensor
- BatterySensor
- SpeedSensor
- TirePressureSensor
- DoorSensor
- SeatbeltSensor

Additional Classes

- Alert
 - AlertManager
 - Dashboard
 - EventLogger
 - VehicleStatistics
 - ThreadManager
-

7. Advanced Features (Bonus)

Teams can implement additional automotive-style features:

Bonus Feature Ideas

Feature	Complexity
Driver profile management	Medium
JSON/XML configuration loading	High
CAN message simulation	High
Adaptive alert prioritization	Medium
Diagnostic Trouble Code (DTC) simulation	High
ECU health monitor	Medium
Performance graph statistics	Medium
OTA update simulation	High
Crash-safe event recording	High
Service-oriented communication model	High

8. Expected Folder Structure

```
Project/
├── include/
│   ├── Sensor.hpp
│   ├── Alert.hpp
│   ├── Logger.hpp
│   └── Dashboard.hpp
├── src/
│   ├── main.cpp
│   ├── Sensor.cpp
│   ├── Alert.cpp
│   ├── Logger.cpp
│   └── Dashboard.cpp
├── logs/
│   └── vehicle_log.txt
├── data/
│   └── config.txt
└── README.md
```

9. Hackathon Execution Plan (15 Hours)

Time	Activity
Hour 1	Requirement analysis and architecture discussion
Hour 2-3	Base class and sensor framework design
Hour 4-5	Alert manager implementation
Hour 6-7	Dashboard and logging implementation
Hour 8-10	STL, templates, file handling integration
Hour 11-12	Multithreading implementation
Hour 13	Exception handling and cleanup
Hour 14	Testing and debugging
Hour 15	Final demo and presentation

10. Evaluation Criteria

Criteria	Weightage
----------	-----------

Criteria	Weightage
C++ Design Quality	25%
OOP Usage	15%
Modern C++ Features	15%
Multithreading & Synchronization	15%
STL & Template Usage	10%
Code Readability & Structure	10%
Innovation & Bonus Features	10%

11. Deliverables

Each team must submit:

- Source code
 - Build instructions
 - README document
 - Demo execution video/screenshots
 - Architecture explanation
 - Thread synchronization explanation
-

12. Constraints

Allowed

- C++11/14/17
- STL
- Threads
- File handling
- Console-based UI

Not Allowed

- External frameworks
 - GUI frameworks
 - Copy-paste internet projects
 - Third-party libraries for core logic
-

13. Real Automotive Mapping

Hackathon Component	Automotive Equivalent
Sensor Framework	MCAL / Sensor Abstraction
Alert Manager	Vehicle Health Manager
Dashboard	Instrument Cluster
Event Logger	Diagnostic Event Manager
Threads	Adaptive AUTOSAR Execution Context
Shared Data Protection	Inter-process synchronization
Runtime Alerts	Safety Monitoring

14. Interview-Oriented Learning Outcome

After completing this hackathon, freshers should confidently explain:

- Why runtime polymorphism is needed
 - RAII benefits in automotive software
 - Thread synchronization challenges
 - STL container selection decisions
 - Why smart pointers improve safety
 - Difference between composition and inheritance
 - Exception-safe design principles
 - Real-time software concurrency challenges
 - Adaptive automotive application architecture
-

15. Final Expected Demo

The final demo should show:

1. Multiple sensor threads running
2. Live sensor updates
3. Alert generation
4. Thread-safe logging
5. File-based event storage
6. Runtime polymorphism in action
7. Modern C++ usage
8. Clean shutdown without memory leaks

16. Optional Industry-Level Extension

If time permits, ask teams to simulate:

- Vehicle entering limp-home mode
- Sensor communication timeout
- Fault recovery handling
- Health monitoring watchdog
- Adaptive warning prioritization

These additions closely resemble real Adaptive AUTOSAR and cockpit-domain controller applications used in modern vehicles.

17. Overview

“Design the system as if it will run inside a real cockpit domain controller ECU. Focus not only on functionality, but also on safe resource handling, scalability, maintainability, and clean modern C++ architecture.”